Dbt 01 Sql Models

Source originale : https://docs.getdbt.com/docs/build/sql-models

Model configurations

Model properties

run

command

ref

function

Building your first models

If you're new to dbt, we recommend that you read a

quickstart guide

to build your first dbt project with models.

dbt's Python capabilities are an extension of its capabilities with SQL models. If you're new to dbt, we recommend that you read this page first, before reading:

"Python Models"

A SQL model is a

select

statement. Models are defined in

.sql

files (typically in your

models

directory):

Each

.sql

file contains one model /

select

statement

The model name is inherited from the filename and must match the

filename

of a model — including case sensitivity. Any mismatched casing can prevent dbt from applying configurations correctly and may affect metadata in

Catalog

.

We strongly recommend using underscores for model names, not dots. For example, use

models/my_model.sql

instead of
models/my.model.sql
.
Models can be nested in subdirectories within the
models
directory.
Starting in
dbt
v1.12, you can use Jinja-style suffixes (
.j2
,
.jinja
,
.jinja2
) on
.sql
files (for example,
my_model.sql.j2
) by enabling the
allow_jinja_file_extensions
flag.
Refer to
How we style our dbt models
for details on how we recommend you name your models.
When you execute the
dbt run
command
, dbt will build this model
data warehouse
by wrapping it in a
create view as
or
create table as
statement.
For example, consider this
customers

model:

models/customers.sql

```
with
customer_orders
as
(
select
customer_id
,
min
(
order_date
)
as
first_order_date
,
max
(
order_date
)
as
most_recent_order_date
,
count
(
order_id
)
as
number_of_orders
from
jaffle_shop
.
orders
group
by
1
```

)

select

customers

.

customer_id

,

customers

.

first_name

,

customers

.

last_name

,

customer_orders

.

first_order_date

,

customer_orders

.

most_recent_order_date

,

coalesce

(

customer_orders

.

number_of_orders

,

0

)

as

number_of_orders

from

jaffle_shop

.

customers

left
join
customer_orders
using
(
customer_id
)
When you execute
dbt run
, dbt will build this as a
view
named
customers
in your target schema:
create
view
dbt_alice
.
customers
as
(
with
customer_orders
as
(
select
customer_id
,
min
(
order_date
)
as
first_order_date
,
max

```
(
order_date
)
as
most_recent_order_date
,
count
(
order_id
)
as
number_of_orders
from
jaffle_shop
.
orders
group
by
1
)
select
customers
.
customer_id
,
customers
.
first_name
,
customers
.
last_name
,
customer_orders
.
first_order_date
```

,
customer_orders
.
most_recent_order_date
,
coalesce
(
customer_orders
.
number_of_orders
,
0
)
as
number_of_orders
from
jaffle_shop
.
customers
left
join
customer_orders
using
(
customer_id
)
Why a
view
named
dbt_alice.customers
? By default dbt will:
Create models as
views
Build models in a target schema you define
Use your file name as the view or
table

name in the database

You can use

configurations

to change any of these behaviors — more on that later.

How can I see the SQL that dbt is running?

To check out the SQL that dbt is running, you can look in:

dbt

:

Within the run output, click on a model name, and then select "Details"

dbt v1

:

The

target/compiled/

directory for compiled

select

statements

The

target/run/

directory for compiled

create

statements

The

logs/dbt.log

file for verbose logging.

Do I need to create my target schema before running dbt?

Nope! dbt will check if the schema exists when it runs. If the schema does not exist, dbt will create it for you.

If I rerun dbt, will there be any downtime as models are rebuilt?

Nope! The SQL that dbt generates behind the scenes ensures that any relations are replaced atomically (i.e. your business users won't experience any downtime).

The implementation of this varies on each warehouse, check out the

logs

to see the SQL dbt is executing.

What happens if the SQL in my query is bad or I get a database error?

If there's a mistake in your SQL, dbt will return the error that your database returns.

$ dbt run

--select
customers
Running with
dbt
=
1.9
.0
Found
3
models,
9
tests,
0
snapshots,
0
analyses,
133
macros,
0
operations,
0
seed files,
0
sources
14
:04:12
|
Concurrency:
1
threads
(
target
=
'dev'
)
14

:04:12
|
14
:04:12
|
1
of
1
START view model dbt_alice.customers
..
[
RUN
]
14
:04:13
|
1
of
1
ERROR creating view model dbt_alice.customers
..
.
[
ERROR
in
0
.81s
]
14
:04:13
|
14
:04:13
|
Finished running
1

view model
in
1
.68s.
Completed with
1
error and
0
warnings:
Database Error
in
model customers
(
models/customers.sql
)
Syntax error: Expected
")"
but got identifier
`
your-info-12345
`
at
[
13
:15
]
compiled SQL at target/run/jaffle_shop/customers.sql
Done.
PASS
=
0
WARN
=
0
ERROR
=

1

SKIP

=

0

TOTAL

=

1

Any models downstream of this model will also be skipped. Use the error message and the

compiled SQL

to debug any errors.

Which SQL dialect should I write my models in? Or which SQL dialect does dbt use?

dbt can feel like magic, but it isn't actually magic. Under the hood, it's running SQL in your own warehouse — your data is not processed outside of your warehouse.

As such, your models should just use the

SQL dialect of your own database

. Then, when dbt wraps your

select

statements in the appropriate

DDL

or

DML

, it will use the correct DML for your warehouse — all of this logic is written in to dbt.

You can find more information about the databases, platforms, and query engines that dbt supports in the

Supported Data Platforms

docs.

Want to go a little deeper on how this works? Consider a snippet of SQL that works on each warehouse:

models/test_model.sql

select

1

as

my_column

To replace an existing

table

, here's an

illustrative

example of the SQL dbt will run on different warehouses (the actual SQL can get much more complicated than this!)

Redshift

BigQuery

Snowflake

-- you can't create or replace on redshift, so use a transaction to do this in an atomic way

begin

;

create

table

"dbt_alice"

.

"test_model__dbt_tmp"

as

(

select

1

as

my_column

)

;

alter

table

"dbt_alice"

.

"test_model"

rename

to

"test_model__dbt_backup"

;

alter

table

"dbt_alice"

.

"test_model__dbt_tmp"

rename

to
"test_model"
commit
;
begin
;
drop
table
if
exists
"dbt_alice"
.
"test_model__dbt_backup"
cascade
;
commit
;
-- Make an API call to create a dataset (no DDL interface for this)!!;
create
or
replace
table
`
dbt-dev-87681
`
.
`
dbt_alice
`
.
`
test_model
`
as
(
select

1
as
my_column
)
;
create
schema
if
not
exists
analytics
.
dbt_alice
;
create
or
replace
table
analytics
.
dbt_alice
.
test_model
as
(
select
1
as
my_column
)
;
Configurations are "model settings" that you can set in your
dbt_project.yml
file,
and
in your model file using a

config

block. Some example configurations include:

Changing the

materialization

that a model uses — a

materialization

determines the SQL that dbt uses to create the model in your warehouse.

Build models into separate

schemas

.

Apply

tags

to a model.

The following diagram shows an example directory structure of a models folder:

models

■■■ staging

■■■ marts

■■■ marketing

Here's an example of a model configuration:

dbt_project.yml

```
name
:
jaffle_shop
config-version
:
2
...
models
:
jaffle_shop
:
# this matches the `name:`` config
+materialized
:
view
# this applies to all models in the current project
```

marts

:

+materialized

:

table

this applies to all models in the `marts/` directory

marketing

:

+schema

:

marketing

this applies to all models in the `marts/marketing/`` directory

models/customers.sql

{{ config

(

materialized

=

"view"

,

schema

=

"marketing"

)

}}

with

customer_orders

as

.

It is important to note that configurations are applied hierarchically — a configuration applied to a subdirectory will override any general configurations.

You can learn more about configurations in the

reference docs

.

What materializations are available in dbt?

dbt ships with five built-in

materializations

:

view

,

table

,

incremental

,

ephemeral

, and

materialized_view

.

Check out the documentation on

materializations

for more information on each of these options.

You can also create your own

custom materializations

. This is an advanced feature of dbt.

What model configurations exist?

You can also configure:

tags

to support easy categorization and graph selection

custom schemas

to split your models across multiple schemas

aliases

if your

view

/

table

name should differ from the filename

Snippets of SQL to run at the start or end of a model, known as

hooks

Warehouse-specific configurations for performance (e.g.

sort

and

dist

keys on Redshift,

partitions
on BigQuery)
Check out the docs on
model configurations
to learn more.
You can build dependencies between models by using the
ref
function
in place of table names in a query. Use the name of another model as the argument for
ref
.
Model
Compiled code in dev
Compiled code in prod
models/customers.sql
with
customers
as
(
select
*
from
{{ ref
(
'stg_customers'
)
}}
)
,
orders
as
(
select
*
from
{{ ref

(
'stg_orders'
)
}}
)
,
.
create
view
dbt_alice
.
customers
as
(
with
customers
as
(
select
*
from
dbt_alice
.
stg_customers
)
,
orders
as
(
select
*
from
dbt_alice
.
stg_orders
)

,
.
)
.
create
view
analytics
.
customers
as
(
with
customers
as
(
select
*
from
analytics
.
stg_customers
)
,
orders
as
(
select
*
from
analytics
.
stg_orders
)
,
.
)

.

dbt uses the

ref

function to:

Determine the order to run the models by creating a dependent acyclic graph (DAG).

The DAG for our dbt project

Manage separate environments — dbt will replace the model specified in the

ref

function with the database name for the

table

(or view). Importantly, this is environment-aware — if you're running dbt with a target schema named

dbt_alice

, it will select from an upstream table in the same schema. Check out the tabs above to see this in action.

Additionally, the

ref

function encourages you to write modular transformations, so that you can re-use models, and reduce repeated code.

You can also document and test models — skip ahead to the section on

testing

and

documentation

for more information.

Are there any example dbt models?

Yes!

Quickstart Tutorial:

You can build your own example dbt project in the

quickstart guide

Jaffle Shop:

A demonstration project (closely related to the tutorial) for a fictional e-commerce store. Refer to

Clone the Jaffle Shop sample project

for clone instructions, or view the

main source code

and

source code using DuckDB

on GitHub.

GitLab:

Gitlab's internal dbt project is open source and is a great example of how to use dbt at scale (

source code

)

dummy-dbt:

A containerized dbt project that populates the Sakila database in Postgres and populates dbt seeds, models, snapshots, and tests. The project can be used for testing and experimentation purposes (

source code

)

Google Analytics 4:

A demonstration project that transforms the Google Analytics 4 BigQuery exports to various models (

source code

,

docs

)

Make Open Data:

A production-grade ELT with tests, documentation, and CI/CD (GHA) about French open data (housing, demography, geography, etc). It can be used to learn with voluminous and ambiguous data. Contributions are welcome (

source code

,

docs

)

If you have an example project to add to this list, suggest an edit by clicking

below.

Can I store my models in a directory other than the `models` directory in my project?

By default, dbt expects the files defining your models to be located in the

models

subdirectory of your project.

To change this, update the

model-paths

configuration in your

dbt_project.yml

file, like so:

dbt_project.yml

model-paths

:

[

"transformations"

]

Can I build my models in a schema other than my target schema or split my models across multiple schemas?

Yes! Use the

schema

configuration in your

dbt_project.yml

file, or using a

config

block:

dbt_project.yml

name

:

jaffle_shop

...

models

:

jaffle_shop

:

marketing

:

+schema

:

marketing

models in the `models/marketing/` subdirectory will use the marketing schema

models/customers.sql

{{

config

(

schema

=

'core'

)

}}

Do ref-able resource names need to be unique?

Within one project: yes! To build dependencies between resources (such as models, seeds, and snapshots), you need to use the

ref

function, and pass in the resource name as an argument. dbt uses that resource name to uniquely resolve the

ref

to a specific resource. As a result, these resource names need to be unique,

even if they are in distinct folders

.

A resource in one project can have the same name as a resource in another project (installed as a dependency). dbt uses the project name to uniquely identify each resource. We call this "namespacing." If you

ref

a resource with a duplicated name, it will resolve to the resource within the same namespace (package or project), or raise an error because of an ambiguous reference. Use

two-argument

ref

to disambiguate references by specifying the namespace.

Those resource will still need to land in distinct locations in the data warehouse. Read the docs on

custom aliases

and

custom schemas

for details on how to achieve this.

How do I remove deleted models from my data warehouse?

If you delete a model from your dbt project, dbt does not automatically drop the relation from your schema. This means that you can end up with extra objects in schemas that dbt creates, which can be confusing to other users.

(This can also happen when you switch a model from being a

view

or

table

, to ephemeral)

When you remove models from your dbt project, you should manually drop the related relations from your schema.

As I create more models, how should I keep my project organized? What should I name my models?

There's no one best way to structure a project! Every organization is unique.

If you're just getting started, check out how we (dbt Labs)

structure our dbt projects

.

If models can only be `select` statements, how do I insert records?

For those coming from an

ETL

(Extract Transform Load) paradigm, there's often a desire to write transformations as

insert

and

update

statements. In comparison, dbt will wrap your

select

query in a

create table as

statement, which can feel counter-productive.

If you wish to use

insert

statements for performance reasons (i.e. to reduce data that is processed), consider

incremental models

If you wish to use

insert

statements since your source data is constantly changing (e.g. to create "Type 2 Slowly Changing Dimensions"), consider

snapshotting your source data

, and building models on top of your snaphots.

Why can't I just write DML in my transformations?

More people know how to write

select

statements, than

DML

, making the transformation layer accessible to more people!

If you write the

DDL

/ DML yourself you can end up getting yourself tangled in problems like:

What happens if the

table

already exists? Or this table already exists as a

view

, but now I want it to be a table?

What if the schema already exists? Or, should I check if the schema already exists?

How do I replace a model atomically (such that there's no down-time for someone querying the table)

What if I want to parameterize my schema so I can run these transformations in a development environment?

What order do I need to run these statements in? If I run a

cascade

does it break other things?

Each of these problems

can

be solved, but they are unlikely to be the best use of your time.

You can test your models, generate documentation, create snapshots, and more!

SQL dialects tend to diverge the most in DML and DDL (rather than in

select

statements) — check out the example

here

. By writing less SQL, it can make a migration to a new database technology easier.

If you do need to write custom DML, there are ways to do this in dbt using

custom materializations

.

How do I specify column types?

Simply cast the column to the correct type in your model:

select

id

,

created::

timestamp

as

created

from

some_other_table

You might have this question if you're used to running statements like this:

create

table

dbt_alice

.

my_table
id
integer
,
created
timestamp
;
insert
into
dbt_alice
.
my_table
(
select
id
,
created
from
some_other_table
)
In comparison, dbt would build this
table
using a
create table as
statement:
create
table
dbt_alice
.
my_table
as
(
select
id
,
created

from

some_other_table

)

So long as your model queries return the correct column type, the table you create will also have the correct column type.

To define additional column options:

Rather than enforcing uniqueness and not-null constraints on your column, use dbt's

data testing

functionality to check that your assertions about your model hold true.

Rather than creating default values for a column, use SQL to express defaults (e.g.

coalesce(updated_at, current_timestamp()) as updated_at

)

In edge-cases where you

do

need to alter a column (e.g. column-level encoding on Redshift), consider implementing this via a

post-hook

.

0